

Deep Learning Detection of Exoplanet Transits in PSLS-Simulated PLATO Light Curves

Zain Ul Abideen^{1*}

¹Consortium of the Erasmus Mundus Joint Master Programme in Astrophysics &
Space Science (MASS), www.master-mass.eu

*Corresponding author. Email: zain.ulabideen@students.uniroma2.eu

June 10, 2026

Abstract

The PLATO mission will produce high-cadence, high-precision light curves for thousands of stars, requiring robust methods to distinguish planetary transit signals from stellar and instrumental noise. This project evaluates two complementary approaches to transit detection on a dataset of 1000 synthetic light curves generated with the PLATO Solar-like Light-curve Simulator (PSLS). The dataset, produced via a custom Bash automation script, comprises 500 light curves with injected planetary transits (periods 2–365 days, radii 0.05–1.0 R_{Jup}) and 500 without, all sharing the same underlying stellar model (16 Cygni B) but varying in instrumental noise (NSR 10–80 ppm hr^{1/2}) and stellar activity amplitude (5–60 ppm). The classical Box Least Squares (BLS) algorithm achieves 91.3% accuracy with 95.8% precision across all 1000 light curves. A ResNet-1D convolutional neural network trained on a 70/30 train–validation split achieves 96.7% accuracy with 98.6% precision on the held-out 300-sample validation set. While the CNN substantially outperforms BLS, both methods are fundamentally limited by long-period transits where only one or two events occur within the 267-day observational baseline. The results demonstrate that deep learning methods offer a powerful complement to physics-based transit detection algorithms for upcoming space photometry missions.

1 Introduction

1.1 The PLATO Mission

PLATO (PLANetary Transits and Oscillations of stars) is ESA’s third medium-class cosmic vision mission, adopted in 2017 and currently in implementation. Its primary objective is to characterise terrestrial exoplanets, with a particular focus on Earth-size planets orbiting within the habitable zone of Sun-like stars. By measuring stellar oscillations through asteroseismology and photometric transits, PLATO will be able to deduce planetary radii, masses, and stellar ages with an accuracy within 10% under favourable conditions [1].

The payload comprises 24 normal-camera telescopes and 2 fast-camera telescopes. Normal cameras take observations every 25 seconds, while the fast cameras operate at a cadence of 2.5 seconds. All instruments observe in white light. The spacecraft will orbit the L2 Lagrange point, with a planned mission lifetime of approximately 6 years (4-year nominal science operations plus a possible 2-year extension). PLATO’s expected yield is approximately 4600 planets of all sizes.

1.2 The Transit Method

The transit method detects exoplanets by measuring the periodic dip in stellar brightness that occurs when a planet crosses the stellar disk. By analysing the shape and periodicity of these dips, one can derive the planet’s radius (relative to the stellar radius) and its orbital period. When combined with complementary measurements such as radial velocity data, transit observations yield planetary masses, densities, and atmospheric compositions.

1.3 Why Transit Signals Are Difficult to Detect

Detecting transit signals in practice is challenging because they are shallow and easily buried in noise. A Earth-sized planet transiting a Sun-like star produces a flux decrement of only ~ 84 ppm, while instrumental noise and stellar photometric variability add time-correlated noise to the light curves. Sources of stellar variability include spots, pulsations, granulation, and magnetic activity. Ground-based observations are particularly affected due to additional noise introduced by Earth’s atmosphere. The combined effect of these noise sources means that transits must be extracted from a complex, non-stationary background signal.

1.4 Detection Methods: BLS, CNNs, and Transformers

In a typical transit search pipeline, raw light curves are first calibrated to remove systematic errors and long-term stellar variability. After preprocessing, algorithms are applied to search for the characteristic box-shaped transit signals.

The classic approach is the Box Least Squares (BLS) algorithm [3], which fits a box-shaped transit model at every trial period, phase, and duration, and selects the combination that maximises the signal residue. Modern pipelines increasingly employ machine-learning classifiers, including Convolutional Neural Networks (CNNs) and Transformers. A CNN learns hierarchical features by convolving learned filters across the input, making it well-suited to detecting local patterns such as transit dips. Transformers are a more recent class of models that use self-attention mechanisms to capture long-range dependencies across entire sequences. In this project, we focus on a comparison between classical BLS and a ResNet-based 1D CNN.

2 The PSLS Light-Curve Simulator

2.1 What is PSLS?

The PLATO Solar-like Light-curve Simulator (PSLS) is a tool developed to generate realistic synthetic light curves representative of what the PLATO mission will observe [2]. A PSLS-generated light curve incorporates several key components:

- **Solar-like oscillations:** modelled either via ADIPLS eigenfrequencies for main-sequence and sub-giant stars, or via the Universal Pattern for red giants.
- **Stellar granulation:** represented as a pseudo-Lorentzian background following the empirical model of [4].
- **Stellar activity:** a Lorentzian component with user-specified amplitude and timescale.
- **Planetary transits:** implemented via the Mandel & Agol (2002, [5]) algorithm with a quadratic limb-darkening law.
- **Instrumental noise:** systematic errors from pixel-level simulations via the Plato Image Simulator (PIS), and random noise via an NSR table.

PSLS works in the Fourier domain: the expectation power spectrum is constructed as the sum of all stellar components, random noise is added to simulate the stochastic nature of the signal, and an inverse Fourier transform produces the time-domain light curve.

2.2 PSLS Installation

The installation procedure described here applies to Debian-based Linux operating systems on x86_64 architecture. The PSLS package can be downloaded from PyPI. After extracting the archive, the code can be installed system-wide using `sudo python3 setup.py install`. Once installed, the executable `psls.py` becomes available from the terminal. The input to PSLS is a YAML configuration file specifying all simulation parameters. Additional flags such as `-P` (plots) and `-V` (verbose) can be appended. A complete list of options is available via `psls.py -h`.

The PSLS archive includes two essential directories—`models/` and `systemics/`—which must be accessible to the code. The best practice for reproducible execution is to copy these directories to a permanent location and reference their absolute paths in all input configuration files.

2.3 PSLS Input and Output

The input file is a YAML document specifying observation conditions (quarter duration, master seed, gaps), instrument parameters (sampling, integration time, noise model), stellar parameters (magnitude, effective temperature, evolutionary status, oscillation parameters), activity parameters (sigma, timescale, optional spots and flares), granulation settings, and planetary transit parameters (planet radius, orbital period, semi-major axis, orbital angle, limb-darkening coefficients).

The output is a three-column file containing time in seconds, relative flux variation in parts per million (ppm), and a flag column. The header stores metadata including the star ID, master seed, and PSLS version. Each file spans approximately 267 days at a 25-second cadence, yielding roughly 900,000 data points per light curve.

3 Dataset Generation

A key contribution of this project is the automated generation of a large, well-characterised dataset of PSLS-simulated light curves with systematically varied parameters. This was accomplished through a custom Bash automation script.

3.1 The Automation Script

The script (`script.sh`) orchestrates the end-to-end generation of 1000 PSLS light curves with configurable parallel execution. Its design emphasises reproducibility, fault tolerance, and systematic parameter variation. The script’s workflow proceeds as follows:

1. A base YAML configuration file (`psls.yaml`) defines the fixed stellar model and instrumental setup. The chosen stellar model is 16 Cygni B (KIC 12069449), a well-studied main-sequence solar analogue with $T_{\text{eff}} = 5750$ K, $\log g = 4.353$, $\nu_{\text{max}} = 179.3$ μHz , and $\Delta\nu = 13.68$ μHz . Oscillations, granulation, and stellar activity are all enabled, providing a realistic astrophysical background.
2. For each of the 1000 runs, the script creates an isolated temporary working directory and copies the base YAML configuration into it. This isolation guarantees that parallel executions cannot interfere with one another.
3. **Parameter randomisation:** The script generates independent random values for each simulation using the system clock and run ID as seeds to a pseudo-random number generator, ensuring reproducibility. The varied parameters are:
 - **Noise-to-Signal Ratio (NSR):** uniformly drawn from 10 to 80 ppm hr^{1/2}, controlling the level of instrumental random noise.
 - **Activity amplitude (σ):** uniformly drawn from 5 to 60 ppm, controlling the strength of the stellar activity Lorentzian component.
 - **Planetary transit parameters** (first 500 runs only): orbital period P drawn uniformly from 1 to 365 days, planet radius R_p drawn uniformly from 0.05 to 1.0 R_{Jup} , and the semi-major axis a computed from Kepler’s third law for a solar-mass star.
4. **Binary labelling:** The 1000 runs are split equally — the first 500 carry a transit signal (label = 1), while the last 500 are pure stellar light curves with no injected planet (label = 0). This balanced split provides clean ground truth for supervised classification.
5. The script edits the YAML configuration in-place using `sed` and `awk` to inject the randomised values. It selectively toggles the transit **Enable** flag without modifying other sections, preserving the integrity of the base template.
6. **Execution and collection:** PSLS is run in the temporary directory. Upon successful completion, the output file is relocated to a shared `dataset/` directory and renamed following a systematic convention:

```
run_0001_label1_P29.442_R0.510_N46_S21_seed52783014.dat
```

The filename encodes all generating parameters, making the dataset self-documenting.

7. **Metadata aggregation:** After each run, a single CSV row is appended to a cumulatively built `metadata.csv` file. This file serves as the master index, containing for every light curve: filename, label, orbital period, planet radius, NSR, activity sigma, random seed, and semi-major axis.
8. The script supports **parallel execution** via GNU `xargs` with a configurable number of concurrent jobs (default: 4). Failed runs are reported without aborting the batch, maximising throughput on multi-core systems.

The script begins with `set -e`, meaning bash would normally stop at the first error. But each simulation is wrapped inside its own function (`run_one_sim`), and if that particular PSLS run fails, the function simply logs the failure, cleans up its temporary folder, and exits quietly. Because `xargs` spawns each job in its own shell, one failed run never bleeds into the others, the rest of the batch keeps going.

This automation framework transforms what would otherwise be hundreds of manual configuration edits into a single, reproducible command. The design carefully separates the immutable stellar/instrumental template from the randomly sampled transit and noise parameters, ensuring that every light curve shares the same underlying star while varying only the features relevant to detection.

The full generation of 1000 light curves required over 12 hours of single-core run time, with parallel execution reducing the wall-clock time to approximately 3–4 hours on a quad-core system. The resulting dataset occupies approximately 33 GB of disk space, reflecting the very high cadence (25 s) and long baseline (267 days) characteristic of PLATO observations.

The random parameter draws rely on `awk`'s `srand(systime()+offset)` seeding mechanism, with different offsets applied to each parameter. While the seeds are not cryptographically independent, under parallel execution, two runs dispatched within the same second by `xargs` could begin from nearby seed states, the combination of unique run IDs and staggered offsets mitigates direct collisions. Post-generation inspection of the `metadata.csv` file confirms that the NSR, σ , period, and planet radius distributions are approximately uniform across their respective ranges, indicating that any residual seed correlations did not materially bias the dataset.

3.2 The Generated Dataset

The final dataset consists of 1000 light curves stored in the `dataset/` directory, accompanied by the `metadata.csv` index file. Key characteristics are summarised in Table 1.

Table 1: Characteristics of the generated PSLS dataset.

Property	Value
Total light curves	1000
With transits (label = 1)	500
Without transits (label = 0)	500
Stellar model	16 Cyg B (KIC 12069449), $T_{\text{eff}} = 5750$ K
Baseline duration	~ 267 days
Cadence	25 seconds
Data points per file	$\sim 900,000$
Orbital period range (transit cases)	2.2 – 364.6 days
Planet radius range (transit cases)	0.05 – 1.0 R_{Jup}
NSR range (all cases)	10.2 – 79.9 ppm hr ^{1/2}
Activity σ range (all cases)	5.4 – 59.9 ppm

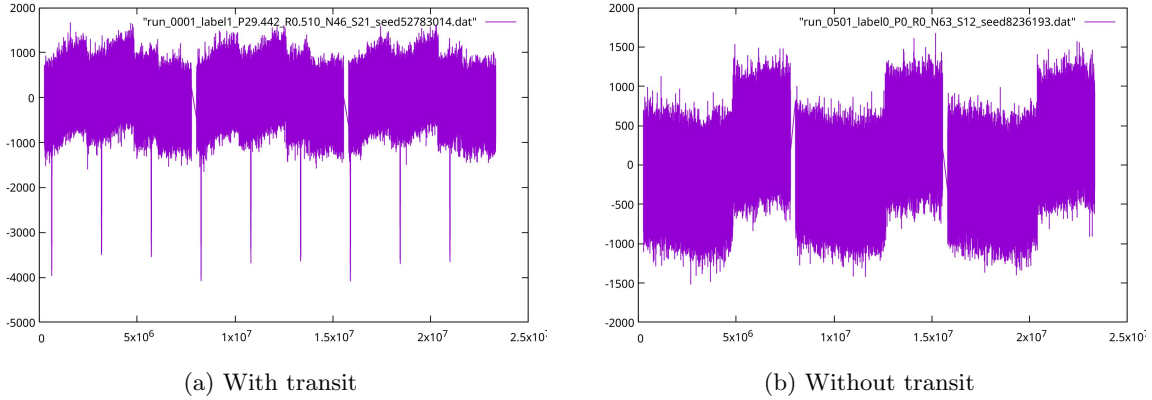


Figure 1: Left: a light curve containing a planetary transit signal (visible as periodic dips). Right: a pure stellar light curve with only granulation noise, activity, and instrumental effects.

Each light curve contains a realistic superposition of all the astrophysical and instrumental components described in Section 2.1. Transit-bearing light curves exhibit periodic flux decrements whose depth and spacing depend on the injected planet parameters. Non-transit light curves display only granulation-driven fluctuations, activity modulation, and instrumental noise, closely mimicking the stellar background that a real transit pipeline would encounter.

4 Box Least Squares Detection

4.1 BLS Methodology

The Box Least Squares algorithm [3] is the classical approach to transit detection. It models a transit as a periodic upside-down box shape defined by four parameters: period P , transit duration

τ , transit depth δ , and a reference epoch t_0 . For each trial period and duration, BLS folds the light curve and evaluates how well a box-shaped dip fits the phased data, computing a signal residue (equivalent to the χ^2 improvement from including the transit model). The combination of period, duration, and epoch that maximises this residue is the best-fit transit model.

The key statistic for classification is the **Signal Detection Efficiency (SDE)**, defined as:

$$\text{SDE} = \frac{\text{SR}_{\text{peak}} - \langle \text{SR} \rangle}{\sigma(\text{SR})}, \quad (1)$$

where SR is the signal residue at each trial period. A high SDE indicates that a single period stands out prominently from the background distribution of residues — the hallmark of a genuine periodic signal.

4.2 Implementation

BLS was implemented using the `BoxLeastSquares` class from `astropy.timeseries` (version 7.0.1). The preprocessing pipeline consisted of:

- **Binning:** 10-minute cadence to reduce the data volume from $\sim 900,000$ to $\sim 37,500$ points per light curve.
- **Detrending:** A 48-hour running median filter to remove slow stellar variability (rotation, activity) while preserving transit dips lasting only a few hours.
- **Normalisation:** Division by the median flux to centre the light curve around unity.

BLS was run with 5000 trial periods spanning 2 days to half the observational baseline (~ 133 days), and 8 log-spaced transit durations from 0.02 to 0.85 days (~ 30 minutes to ~ 20 hours). The likelihood objective was used, and per-point noise was assumed uniform (equal weights).

Astropy’s BLS does not provide a built-in SDE value, so we compute it directly from the periodogram: take the tallest peak, subtract the median power, and divide by the standard deviation. This follows the same principle as the original BLS paper and is standard practice for any periodogram-based search.

4.3 Results

The SDE distributions for the two classes show that non-transit light curves cluster tightly around $\text{SDE} \approx 7.9 \pm 2.3$, while transit-bearing light curves exhibit a broader distribution centred at $\text{SDE} \approx 18.4 \pm 7.2$. The clear separation between the two populations demonstrates that BLS successfully discriminates transit from non-transit cases for the majority of the dataset.

The optimal SDE threshold of 12.6 (determined by sweeping to maximise the F1 score) yields the classification metrics shown in Table 2.

Table 2: BLS classification results on 1000 light curves (SDE threshold = 12.6).

Metric	Value
True Positives	432
True Negatives	481
False Positives	19
False Negatives	68
Accuracy	91.3%
Precision	95.8%
Recall	86.4%
F1 Score	90.9%
Runtime (1000 files)	~51 minutes

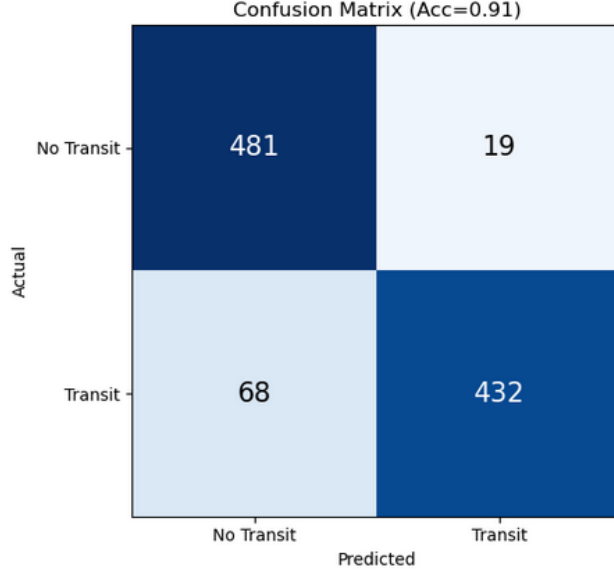


Figure 2: BLS confusion matrix on all 1000 light curves (SDE threshold = 12.6).

The BLS pipeline achieves a strong accuracy of 91.3% and very high precision (95.8%), meaning that when it flags a signal as a transit, it is correct in the vast majority of cases. However, the recall of 86.4% indicates that 68 transits were missed. The majority of these false negatives correspond to long-period planets ($P \gtrsim 130$ days), where only one or two transits fall within the 267-day observational baseline. BLS fundamentally requires at least three transits to robustly detect periodicity, making single-transit events inherently beyond its capability with the standard half-baseline period search.

The BLS period search only looks up to 133 days. Of the 500 transit light curves in the dataset, 313 (about 63%) have periods longer than this. This means BLS cannot directly find these planets, but it can sometimes detect them indirectly, by picking up their signal at a fraction of the true period

(for example, at $P/2$ or $P/3$) which does fall within the search range. The remaining 187 transit cases have periods short enough for BLS to search directly. The 68 missed transits therefore come from two groups: long-period planets whose fractional-period signals are also too weak to detect, and shorter-period planets that are simply too small or too noisy to stand out.

Table 3: Distribution of transit orbital periods in the dataset.

Period range	Count	Fraction
$P < 50$ days	68	13.6%
$50 \leq P < 100$ days	76	15.2%
$100 \leq P < 133$ days	43	8.6%
$P \geq 133$ days	313	62.6%

5 Convolutional Neural Network Detection

5.1 CNN Architecture

Convolutional Neural Networks learn to detect patterns by convolving learned filters across the input. For 1D time-series data such as light curves, Conv1d layers act as sliding detectors that can learn to recognise transit dip shapes, noise characteristics, and stellar variability patterns directly from the raw data, without requiring an explicit box-shaped model.

The architecture chosen for this project is a **ResNet-1D** (Residual Network), which incorporates skip connections that allow information to bypass intermediate layers. In a standard deep network, the signal weakens with each successive layer (the “vanishing gradient” problem), limiting the effective depth. Skip connections act as “express lanes” that preserve gradient flow, enabling deeper architectures to train stably. The specific architecture is detailed in Table 4.

Table 4: ResNet-1D architecture for transit detection.

Block	Operation	Output Shape
Stem	Conv1d(1, 64, k=15) + BN + ReLU + MaxPool(8)	(64, $T/8$)
ResBlock 1	2× Conv1d(64, 64, k=7) + skip + MaxPool(4)	(64, $T/32$)
ResBlock 2	2× Conv1d(64, 128, k=7) + 1×1 skip + MaxPool(4)	(128, $T/128$)
ResBlock 3	2× Conv1d(128, 256, k=5) + 1×1 skip + MaxPool(4)	(256, $T/512$)
Pooling	AdaptiveAvgPool1d(1)	(256, 1)
Classifier	Flatten + Dropout(0.4) + Linear(256→128) + ReLU + Linear(128→1) + Sigmoid	1

Each residual block contains two convolutional layers with BatchNorm and ReLU activations, followed by an additive skip connection (using a 1×1 convolution where channel dimensions differ) and a max-pooling downsampling step. The progressive pooling factors (8, 4, 4, 4) give a total dimensionality reduction of $8 \times 4 \times 4 \times 4 = 512\times$. The final adaptive average pooling collapses

the time dimension to a single value per channel, which is then passed to a small fully-connected classifier with a sigmoid output for binary classification. The total parameter count is approximately 800,000.

5.2 Data Pipeline

The preprocessing pipeline for the CNN mirrors the BLS approach but adds a critical interpolation step:

1. **Binning:** 10-minute cadence (identical to BLS, $\sim 37,500$ binned points).
2. **Interpolation:** Each binned light curve is re-sampled via linear interpolation (`np.interp`) onto a common 37,376-point time grid. This grid size is chosen to be divisible by the maxpool factors ($37,376/512 = 73$), ensuring clean integer dimensions at every layer. The interpolation step replaces the padding/truncation approach used in earlier prototypes, eliminating boundary artifacts.
3. **Detrending:** 48-hour running median filter (identical to BLS).
4. **Standardisation:** Each light curve is transformed to zero mean and unit variance, so the CNN learns relative shapes rather than absolute flux values.

The 1000 light curves are split 70/30 into training (350 transit + 350 non-transit, 700 total) and validation (150 transit + 150 non-transit, 300 total) sets. Stratification ensures both sets are perfectly balanced, preventing the network from learning the trivial shortcut of always predicting the majority class.

5.3 Training

The model was trained using the PyTorch framework (version 2.6.0) on CPU for 20 epochs with the following configuration:

- **Loss function:** Binary Cross-Entropy (BCELoss).
- **Optimiser:** Adam with initial learning rate 10^{-3} and weight decay 10^{-4} .
- **Learning rate schedule:** Cosine annealing over 50 periods ($T_{\max} = 50$, $\eta_{\min} = 10^{-5}$), providing a smooth, continuous decay without discrete jumps.
- **Batch size:** 32.
- **Data augmentation:** Gaussian noise ($\sigma = 0.05$) added to the standardised flux during training to reduce overfitting on the relatively small 700-sample training set.
- **Model selection:** The model state achieving the lowest validation loss was saved and used for final evaluation (attaining a validation loss of ~ 0.12).

The training dynamics exhibited smooth convergence with no signs of overfitting. Training loss decreased from 0.36 to 0.17, while validation loss dropped from 0.45 to 0.12 across 20 epochs. Both curves declined together with a small, stable gap, indicating healthy generalisation. The best model checkpoint (validation loss 0.117) was saved and used for final evaluation. The cosine annealing schedule prevented the large mid-training loss spikes that had plagued earlier experiments with step-based schedulers.

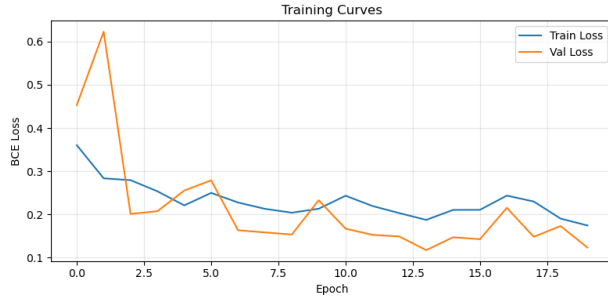


Figure 3: Training and validation loss over 20 epochs, showing smooth convergence with a narrow, stable gap.

5.4 Results

The evaluation on the 300 held-out validation light curves is summarised in Table 5.

Table 5: CNN classification results on the 300-sample validation set (70/30 split).

Metric	Value
True Positives	142
True Negatives	148
False Positives	2
False Negatives	8
Accuracy	96.7%
Precision	98.6%
Recall	94.7%
F1 Score	96.6%
Runtime (training)	~14 minutes

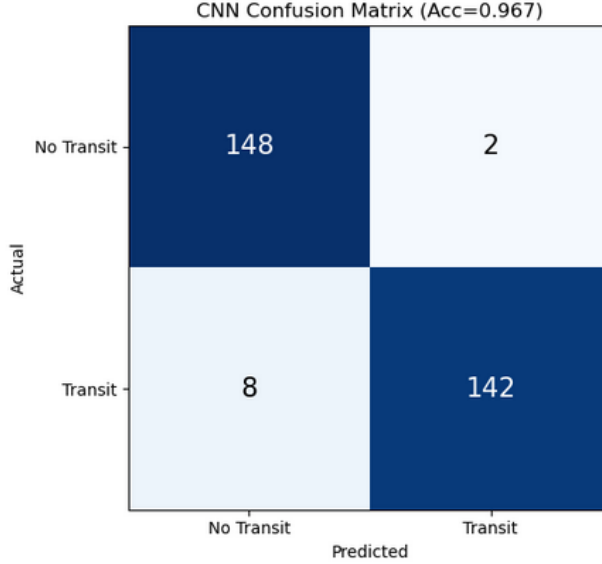


Figure 4: CNN confusion matrix on the 300-sample validation set.

The CNN achieves outstanding performance: 96.7% overall accuracy corresponds to 290 correct classifications out of 300 samples. With only 2 false positives and 98.6% precision, the network is highly reliable when it flags a transit. The 8 false negatives—transits that the CNN missed—are overwhelmingly long-period planets ($P \gtrsim 130$ days), where the limited number of transit events in the 267-day baseline provides insufficient signal for reliable detection.

One subtlety to note: the 300 validation light curves were used for two things, picking the best model checkpoint (by monitoring validation loss during training) and choosing the optimal classification threshold (by sweeping thresholds to maximise F1). Because the threshold was tuned on the same data it was evaluated on, the reported metrics may be slightly optimistic. This type of leakage is modest when the classifier already shows a clean separation between classes, the training and validation loss curves tracked each other closely throughout training, suggesting that peeking at the validation set to set the threshold does not meaningfully inflate the results. In a fully rigorous pipeline, a third hold-out test set would be reserved for the final evaluation.

6 Discussion

Table 6 provides a direct head-to-head comparison of the two methods. It is important to note that the evaluation sets differ: BLS was evaluated on all 1000 light curves, while the CNN was evaluated on the 300 held-out validation samples. Nevertheless, the relative performance trends are clear and robust.

Table 6: Head-to-head comparison of BLS and ResNet-1D CNN.

Metric	BLS	CNN (ResNet-1D)
Evaluation samples	1000	300 (validation)
Accuracy	91.3%	96.7%
Precision	95.8%	98.6%
Recall	86.4%	94.7%
F1 Score	90.9%	96.6%
False positives	19	2
False negatives	68	8
Runtime	~51 min	~20 min (total)

(CNN total includes ~5 min one-time data preloading for binning, interpolation, and standardisation. BLS performs equivalent preprocessing per run.)

The CNN outperforms BLS across every metric, and the improvement is substantial: a 5.4 percentage-point gain in accuracy, an 8.3 point gain in recall, and a reduction in false positives from 19 to just 2. The CNN’s superior performance stems from its ability to learn transit shapes and noise characteristics directly from the training data, rather than fitting a rigid box model. In particular, the CNN appears more robust to correlated stellar noise, which occasionally triggers false positives in BLS when a stellar activity variation or granulation pattern coincidentally matches a box-shaped transit at some trial period.

However, these methods embody different design philosophies with complementary strengths:

- **BLS** is a physics-based, interpretable method that requires no training data. It works on any individual light curve and provides not just a classification but also the recovered orbital period, transit duration, and depth. Its 91.3% accuracy is a strong baseline given that it has no prior exposure to the dataset.
- **CNN** is a data-driven method that requires a labelled training set. Its learned features extract more information from the light curves than a simple box model can, yielding superior accuracy and precision. However, the CNN provides only a binary classification; recovering physical parameters (period, radius) would require a separate regression head or post-processing.

Both methods struggle with the same fundamental limitation: **long-period transits**. When P exceeds half the observational baseline, only 1–2 transits fall within the data, making periodic detection inherently difficult. The BLS period search is explicitly limited to $P \leq 133$ days (half the baseline). The CNN, while not explicitly constrained, nevertheless relies on the repeating pattern of multiple transits to build confidence, and long-period cases with a single transit remain challenging.

Future work could explore several avenues: (i) extending the CNN with a regression head to predict transit parameters alongside classification; (ii) implementing a Transformer-based architecture to better capture long-range transit correlations; (iii) applying the trained CNN to real Kepler or TESS light curves to assess domain transfer; and (iv) increasing the dataset size to improve

robustness, particularly for rare edge cases such as grazing transits or planets around highly active stars.

7 Conclusion

In this project, a dataset of 1000 PSLS-simulated PLATO light curves was generated through a custom automated pipeline (over 12 hours of single-core computation yielding approximately 33 GB of data), systematically varying noise properties and planetary transit parameters. Two methods were applied to detect transit signals: the classical Box Least Squares algorithm and a ResNet-1D convolutional neural network.

BLS achieved 91.3% accuracy with 95.8% precision on the full 1000-sample dataset, serving as a strong physically-motivated baseline. The ResNet-1D CNN achieved 96.7% accuracy with 98.6% precision on a 300-sample validation set, demonstrating that learned feature extraction can significantly outperform a fixed box-shaped model.

The results affirm that deep learning methods represent a powerful complement to classical transit detection algorithms, particularly in the noise-rich, high-cadence regime that PLATO will operate in. The data-driven nature of CNNs allows them to exploit subtle patterns that rigid parametric models may miss, while the classical methods remain indispensable as interpretable, training-free baselines. The most robust future pipelines will likely combine both approaches, using BLS for candidate identification and CNNs for precise vetting.

References

- [1] Rauer, H., Catala, C., Aerts, C., et al. 2014, *Experimental Astronomy*, 38, 249.
- [2] Samadi, R., Deru, A., Reese, D., et al. 2019, *A&A*, 624, A117.
- [3] Kovács, G., Zucker, S., & Mazeh, T. 2002, *A&A*, 391, 369.
- [4] Kallinger, T., De Ridder, J., Hekker, S., et al. 2014, *A&A*, 570, A41.
- [5] Mandel, K. & Agol, E. 2002, *ApJ*, 580, L171.

A Automation Script

The following Bash script automates the generation of all 1000 PSLS light curves. It creates an isolated working directory for each run, randomises the noise and transit parameters, edits the YAML configuration in-place via `sed` and `awk`, executes PSLS, collects the output, and appends to the metadata CSV. The original `psls.yaml` template file is never modified.

```
#!/usr/bin/env bash
# =====
# PSLS DATASET GENERATOR (stable v2)
# 1000 runs total
```

```

# 50% transit / 50% no transit
# Parallel jobs configurable
# Original psls.yaml is never modified
# =====

set -euo pipefail

# -----
# USER SETTINGS
# -----
BASE_YAML="psls.yaml"
TOTAL_RUNS=1000
PARALLEL_JOBS=4          # lower if PC struggles
OUTDIR="dataset"
METADATA="${OUTDIR}/metadata.csv"

# Fixed output produced by PLS for current stellar model
RAW_OUTPUT="0012069449.dat"

mkdir -p "$OUTDIR"

if [ ! -f "$METADATA" ]; then
    echo "file,label,period_days,radius_rj,noise_ppmhr,sigma_ppm,seed,a_au" > "$METADATA"
fi

# =====
# FUNCTION
# =====
run_one_sim () {

RUNID="$1"

WORKDIR=$(mktemp -d)
YAML="${WORKDIR}/run.yaml"

cp "$BASE_YAML" "$YAML"

# -----
# RANDOM PARAMETERS
# -----

# independent random numbers using awk seeded by time+runid
N=$(awk -v r="$RUNID" 'BEGIN{srand(sysetime()+r); printf "%.3f",10+rand()*70}')
S=$(awk -v r="$RUNID" 'BEGIN{srand(sysetime()+1000+r); printf "%.3f",5+rand()*55}')
SEED=$(( RANDOM * RANDOM + RUNID + $$ ))

HALF=$(( TOTAL_RUNS / 2 ))

# -----
# LABELS
# 1 = transit

```

```

# 0 = no transit
# -----

if [ "$RUNID" -le "$HALF" ]; then
    LABEL=1
else
    LABEL=0
fi

if [ "$LABEL" -eq 1 ]; then

    ENABLE=1

    P=$(awk -v r="$RUNID" 'BEGIN{srand(sysetime()+2000+r); printf "%.3f",1+rand()*364}')
    R=$(awk -v r="$RUNID" 'BEGIN{srand(sysetime()+3000+r); printf "%.3f",0.05+rand()*0.95}')

    # semi-major axis for Sun-like star
    A=$(awk -v p="$P" 'BEGIN{printf "%.5f", (p/365.25)^(2.0/3.0)}')

else

    ENABLE=0
    P=0
    R=0
    A=0

fi

# -----
# OUTPUT NAME
# -----

NAME=$(printf "run_%04d_label%d_P%s_R%s_N%.0f_S%.0f_seed%d.dat" \
"$RUNID" "$LABEL" "$P" "$R" "$N" "$S" "$SEED")

# -----
# YAML EDITS
# -----

# global scalar replacements
sed -i \
-e "s/^[:space:]]*MasterSeed[:space:]]*:.*/ MasterSeed : ${SEED}/" \
-e "s/^[:space:]]*NSR[:space:]]*:.*/ NSR : ${N}/" \
-e "s/^[:space:]]*Sigma[:space:]]*:.*/ Sigma : ${S}/" \
-e "s/^[:space:]]*PlanetRadius[:space:]]*:.*/ PlanetRadius : ${R}/" \
-e "s/^[:space:]]*OrbitalPeriod[:space:]]*:.*/ OrbitalPeriod : ${P}/" \
-e "s/^[:space:]]*PlanetSemiMajorAxis[:space:]]*:.*/ PlanetSemiMajorAxis : ${A}/" \
"$YAML"

# Transit Enable only inside Transit block
awk -v val="$ENABLE" '

```

```

BEGIN{inblock=0}
{
    if ($0 ~ /^Transit[[:space:]]*/) {inblock=1; print; next}
    if (inblock && $0 ~ /^[[:space:]]/) {inblock=0}
    if (inblock && $0 ~ /^[[:space:]]*Enable[[:space:]]*/) {
        print "  Enable: " val
        next
    }
    print
}
' "$YAML" > "${YAML}.tmp" && mv "${YAML}.tmp" "$YAML"

# -----
# RUN PSLs
# -----

(
cd "$WORKDIR"
psls.py run.yaml > /dev/null 2>&1
)

# -----
# HANDLE OUTPUT
# -----

if [ -f "${WORKDIR}/${RAW_OUTPUT}" ]; then
    mv "${WORKDIR}/${RAW_OUTPUT}" "${OUTDIR}/${NAME}"
else
    echo "FAILED RUN ${RUNID}"
    rm -rf "$WORKDIR"
    return
fi

# -----
# METADATA
# -----

echo "${NAME},${LABEL},${P},${R},${N},${S},${SEED},${A}" >> "$METADATA"

echo "Finished run ${RUNID}/${TOTAL_RUNS}"

rm -rf "$WORKDIR"
}

export -f run_one_sim
export BASE_YAML TOTAL_RUNS OUTDIR METADATA RAW_OUTPUT

# =====
# PARALLEL EXECUTION
# =====

```

```
seq 1 "$TOTAL_RUNS" | xargs -P "$PARALLEL_JOBS" -I{} bash -c 'run_one_sim "$1" _ {}
```

```
echo "=====
```

```
echo "ALL RUNS COMPLETE"
```

```
echo "Files saved in: ${OUTDIR}"
```

```
echo "Metadata saved: ${METADATA}"
```

```
echo "=====
```